

CN Assignment 4

[GitHub](#)

Vimansh Mahajan

2022572

Question 1)

a) the maximum expected value (theoretical) = **7Mbps**

Bottleneck Bandwidth:

- Throughput is determined by the **slowest link** in the path.
- The bottleneck is $N1 \rightarrow N2$, with a bandwidth of **7 Mbps**.

It is maximum because:

- Data cannot travel faster than the slowest link in the path.
- Even if $N0 \rightarrow N1$ can handle 10 Mbps, the throughput will be limited to $N1 \rightarrow N2$'s capacity of 7 Mbps.

b)

$$d_{trans} = d_{trans_n0} + d_{trans_n1} = 1460 * 8 / (7 * 10^6) + 1460 * 8 / (10 * 10^6) = 2.8365714 \text{ms}$$

$$d_{trans_ACK} = 0 \text{ ms (assuming it is negligible compared to the packet size).}$$

$$d_{prop_n0n1} = 100 \text{ ms}$$

$$d_{prop_n1n2} = 10 \text{ ms}$$

$$RTT = d_{trans_packet} + d_{trans_ACK} + 2 * d_{prop_n0n1} + 2 * d_{prop_n1n2} = 2.836 + 0 + 200 + 20 = 222.8365714 \text{ ms}$$

$$BDP = \text{bottleneck bandwidth} \times RTT = 7 \times 10^6 \times 222.8365714 \times 10^{-3} = \mathbf{1559855.9998 \text{ bits}}$$

In terms of packets: $1559855.9998 / (1460 * 8) = \mathbf{133.549 \text{ packets}}$ (it can be rounded down to **133 packets** to maintain complete packets)

c)

```
vimansh_mahajan@Vimansh-PC:~/ns-3-dev$ tshark -r tcp-example-2-0.pcap -T fields -e frame.time_epoch -e ip.src -e ip.dst -e tcp.port -e tcp.seq -e tcp.len -e tcp.ack > receiver_data.txt
vimansh_mahajan@Vimansh-PC:~/ns-3-dev$ python3 calculate_throughput.py
Total Bytes Transferred: 2746700 bytes
Transfer Duration: 8.86 seconds
Average Throughput: 2.48 Mbps
```

```
import pandas as pd

# Load the data from receiver_data.txt
data = []
with open('receiver_data.txt', 'r') as file:
    for line in file:
        fields = line.strip().split('\t')
        if len(fields) == 7:
            data.append(fields)

# Define column names for the extracted data
columns = ['timestamp', 'src_ip', 'dst_ip', 'port', 'seq_num', 'tcp_len', 'ack']
df = pd.DataFrame(data, columns=columns)

# Convert necessary columns to appropriate data types
df['timestamp'] = df['timestamp'].astype(float)
df['tcp_len'] = df['tcp_len'].astype(int)

# Calculate total bytes transferred and the duration of the transfer
total_bytes = df['tcp_len'].sum()
start_time = df['timestamp'].min()
end_time = df['timestamp'].max()
duration = end_time - start_time # In seconds

# Calculate average throughput in bytes per second
average_throughput_bps = total_bytes / duration
```

```
# Convert throughput to Mbps for easier interpretation
average_throughput_mbps = (average_throughput_bps * 8) / 1_000_000

# Print the results
print(f"Total Bytes Transferred: {total_bytes} bytes")
print(f"Transfer Duration: {duration:.2f} seconds")
print(f"Average Throughput: {average_throughput_mbps:.2f} Mbps")
```

Total Bytes Transferred: 2746700 bytes

Transfer Duration: 8.86 seconds

Average Throughput: 2.48 Mbps

The average throughput comes out to be 2.48 Mbps when measured using tshark:

- ./ns3 run scratch/tcp_example.cc
- tshark -r tcp-example-2-0.pcap -T fields -e frame.time_epoch -e ip.src -e ip.dst -e tcp.port -e tcp.seq -e tcp.len -e tcp.ack > receiver_data.txt
- vimansh_mahajan@Vimansh-PC:~/ns-3-dev\$ python3 calculate_throughput.py

Formula: (Total_bytes x 8) / 8.86 seconds

d)

The maximum achievable throughput was **7 Mbps**, assuming no network congestion, packet loss, or other performance-degrading factors.

Calculated Throughput: **2.48 Mbps**

The calculated throughput, **2.48 Mbps**, is significantly lower than the theoretical maximum of **7 Mbps**. There are several possible reasons for this difference:

1. TCP Overhead:

- TCP involves various overheads such as packet headers (IP, TCP), flow control mechanisms, acknowledgements, and retransmissions. These overheads consume bandwidth, reducing the available throughput for the actual data payload.

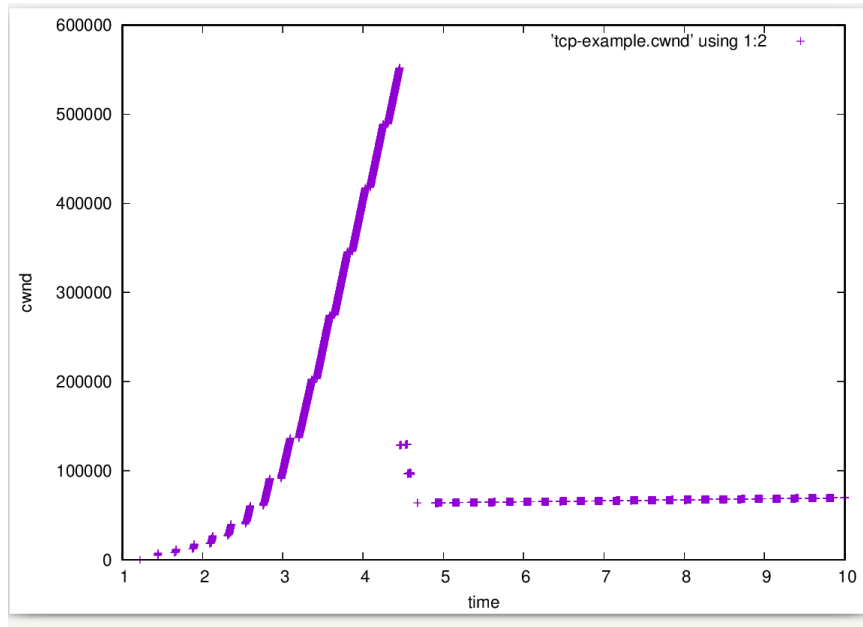
2. Congestion and Packet Loss:

- Even with minimal congestion, some packets might be dropped, leading to retransmissions and retransmission delays. This would lower the effective throughput.

3. Queueing Delays:

- Even though the simulation has a small queue size, queueing delays at the nodes (such as N1 and N2) can cause packets to be delayed or arrive later than expected, reducing throughput.

e)



Exponential Growth at the Start (Slow Start):

- The congestion window (cwnd) increases exponentially during the initial phase (from $t=1$ to $t=3$). This behavior is consistent with the *slow start* phase of TCP, where cwnd doubles for every round-trip time (RTT) until it reaches the slow-start threshold (ssthresh).

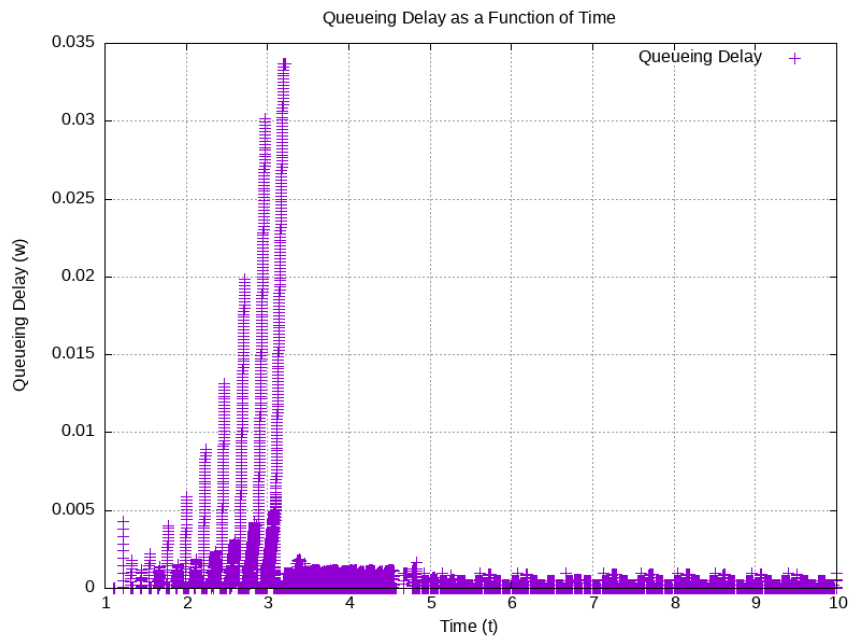
Linear Growth Afterward (Congestion Avoidance):

- The growth becomes linear beyond a certain point ($t=3$ onwards). This reflects the *congestion avoidance* phase, where cwnd increases constantly (roughly one MSS per RTT).

Flat Plateau (Packet Loss or Reaching Capacity):

- Around $t=5$, the cwnd remains constant

f)



Initial Phase (0–1.5 seconds): The queueing delay remains consistently low, indicating smooth network traffic with minimal congestion.

Sudden Spike (1.5–3.5 seconds): A sharp increase in queueing delay occurs, suggesting congestion or heavy traffic buildup in the network.

Stabilisation (after 3.5 seconds): The queueing delay decreases and stabilises at lower values, reflecting successful congestion management and a return to steady network performance.

g)

1. Slow Start Phase (Exponential Growth in cwnd)

- In the **cwnd plot**, the congestion window increases exponentially in the initial phase (up to approximately $t=3$).
- As the sender increases the amount of data sent, more packets enter the network. This leads to **higher queueing delay**, as reflected by the spikes in the **queueing delay plot** around the same time ($t=2$ to $t=3$).
- The spikes occur because the queue at the bottleneck link fills up due to the rapid increase in traffic.

2. Transition to Congestion Avoidance (Linear Growth in cwnd)

- After reaching the slow-start threshold, the **cwnd growth slows down to a linear rate**, which reduces the pressure on the network.

- Correspondingly, the queuing delay stabilizes and diminishes after the peak at $t=3$. This is because TCP reduces its sending rate to avoid overwhelming the network.

3. Plateau in cwnd

- Around $t=5$, the **cwnd plot flattens**, indicating that TCP has reached a stable sending rate, dictated by the network's capacity or congestion control.
- In the queuing delay plot, we see very low queuing delay during this period ($t=5$ onwards). This stability indicates that the sender and receiver have achieved a balance where the network queue is not overfilled.

4. Conclusion

- **Queuing Delay Peaks Signal Congestion:** The spikes in queuing delay indicate congestion, which TCP detects using packet loss or increased RTT. This feedback is used to adjust cwnd.
- **TCP's Congestion Control Stabilizes Queuing Delay:** Once TCP adjusts its cwnd, it reduces the queuing delay, as the network operates closer to its capacity without excessive queuing.
- **Cause-Effect Relationship:** The queuing delay is directly caused by how TCP adjusts the sending rate (via cwnd), and the feedback from queuing delay or packet loss shapes future adjustments to cwnd.

Question 2)

(a) Set queue size = 1000p

```
//set qsize for N0 and N1
pointToPoint.SetQueue ("ns3::DropTailQueue", "MaxSize", StringValue ("1000p"));
NetDeviceContainer devices;
devices = pointToPoint.Install (n0n1);
```

```
//set qsize for N1 and N2
pointToPoint2.SetQueue ("ns3::DropTailQueue", "MaxSize", StringValue ("1000p"));
NetDeviceContainer devices2;
devices2= pointToPoint2.Install (n1n2);
```

```
vimansh_mahajan@Vimansh-PC:~/ns-3-dev$ python3 calculate_throughput.py
Total Bytes Transferred: 2746700 bytes
Transfer Duration: 8.86 seconds
Average Throughput: 2.48 Mbps
```

The average throughput is still 2.48Mbps

Possible reasons:

1. Bandwidth Limitation

- Throughput is often limited by the link bandwidth rather than the queue size. If the link capacity is saturated, increasing the queue size won't improve throughput, as the bottleneck is at the link itself.

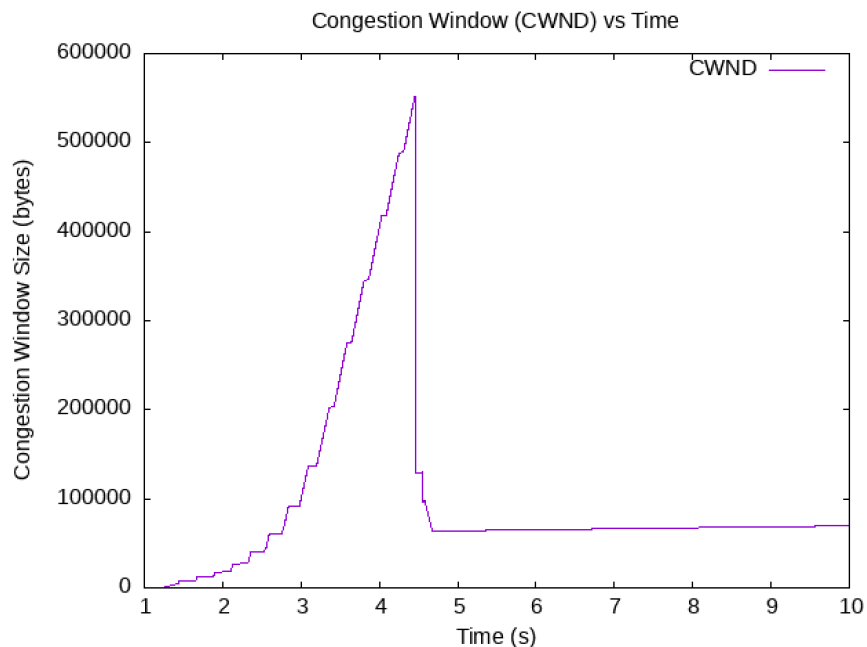
2. Congestion Control Protocols

- In TCP, the throughput is limited by the congestion window and round-trip time. A larger queue size doesn't necessarily affect the congestion window, as it depends on factors like packet loss and delay.

3. Traffic Pattern

- The traffic rate from the source is not high enough to utilize the increased queue size. The larger queue size won't impact throughput if the source is not sending packets fast enough to fill the queue.

(b)



Slow Start Phase (1 to ~3 seconds):

- The CWND increases exponentially, which is characteristic of TCP's slow start phase. During this period, the sender rapidly probes the network for available bandwidth by doubling the CWND after every round-trip time (RTT).

Transition to Congestion Avoidance (~3 to ~4 seconds):

- The rate of CWND growth appears to slow down, indicating a transition from the slow start phase to congestion avoidance. In this phase, CWND grows linearly, adding one maximum segment size (MSS) per RTT.

Congestion Event (~4 seconds):

- The sharp drop in CWND at approximately 4 seconds suggests a congestion event, likely caused by packet loss or delay exceeding TCP's timeout threshold. This event triggers TCP's congestion control mechanism, such as a fast retransmit or timeout, causing the CWND to drop significantly.

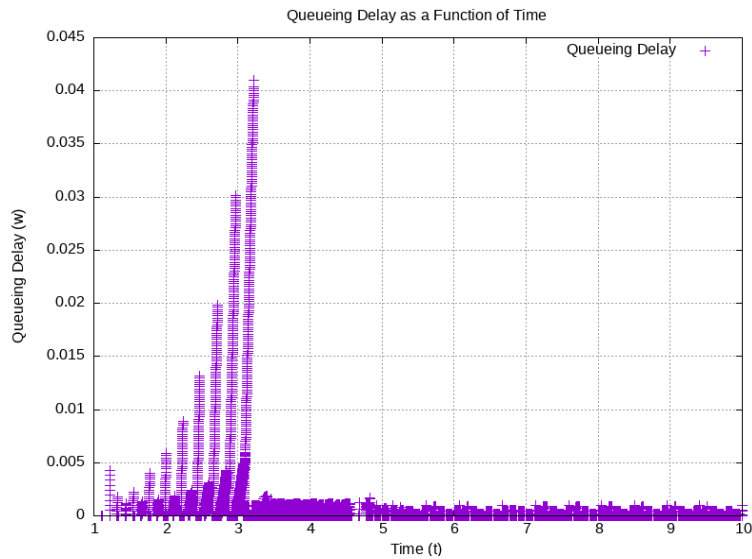
Recovery Phase (~4 to ~5 seconds):

- After the congestion event, the CWND stabilizes at a smaller size, reflecting TCP's attempt to recover from congestion by re-entering a slower growth phase.

Stability (~5 to 10 seconds):

- The CWND remains relatively flat for the rest of the time, indicating a steady-state condition where TCP has adjusted to the available network capacity and is avoiding further congestion.

(c)



Spike in Queueing Delay:

- A significant spike in queueing delay occurs around $t = 3$ seconds, reaching its peak. This indicates a momentary surge in traffic or congestion at this point, potentially due to a burst in incoming packets exceeding the queue's capacity.

Stabilization After Peak:

- After the peak at $t \approx 3$ seconds, the queueing delay quickly diminishes and stabilizes near zero. This suggests that the network returns to a steady state due to congestion control mechanisms or reduced incoming traffic.

Low Delay Before and After Peak:

- The queueing delay is minimal and nearly constant before $t = 3$ seconds and after $t = 4$ seconds, indicating efficient handling of packets during these periods with minimal congestion.

(d)

Observations:

1. Queue Length 50 (First Plot):

- **Behavior:**
 - The congestion window size (CWND) increases rapidly until approximately $t=4$.
 - A sharp drop occurs after reaching the peak, followed by stabilization at a lower value.

2. Queue Length 1000 (Second Plot):

- **Behavior:**
 - The congestion window grows similarly until approximately $t=4$, where it peaks.
 - A drop occurs after reaching the peak, followed by stabilization at a lower value.

Key Observations and Insights:

There are no significant and visible differences between both the cwnd plots

1. **Peak CWND Size:**
 - The CWND peak in both cases is similar.
 2. **Post-Peak Behavior:**
 - In both graphs, the peak drops to a lower value and stays almost constant at that value.
 3. **Stabilization Point:**
 - The slope is constant after the fall of peak in both the cases.
- The bottleneck in the network may be caused due to reasons like bottleneck bandwidth and not due to queue size.
 - The change in CWND by algorithms like AIMD is not directly dependent on the queue size as much as other factors. In other words, the queue may not be full even with 50p often, which causes packet drops therefore lesser impact of changed queue size.

Question 3)

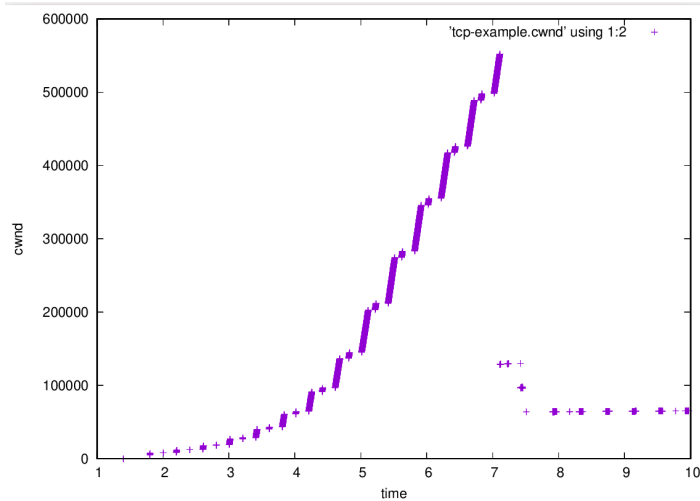
a)

```
vimansh_mahajan@Vimansh-PC:~/ns-3-dev$ python3 calculate_throughput.py
Total Bytes Transferred: 1541772 bytes
Transfer Duration: 8.77 seconds
Average Throughput: 1.41 Mbps
```

Average Throughput: 1.41Mbps

- **Reasons:**
 - Increased RTT reduces TCP throughput as it grows the congestion window more slowly
- When delays increase, TCP reduces its sending rate because it interprets longer delays as a sign of congestion. This can result in lower throughput, even if bandwidth is available.
- TCP depends on fast feedback from acknowledgement packets to adjust its sending rate. The increased delay in your setup slowed down this feedback loop, causing TCP to perform less effectively.

b)



Slow Start Phase:

In the initial phase (approximately from time = 1 to 6), the cwnd increases exponentially. This behaviour aligns with the **slow start phase** of TCP congestion control, where the congestion window size doubles every round-trip time (RTT) until it reaches the threshold.

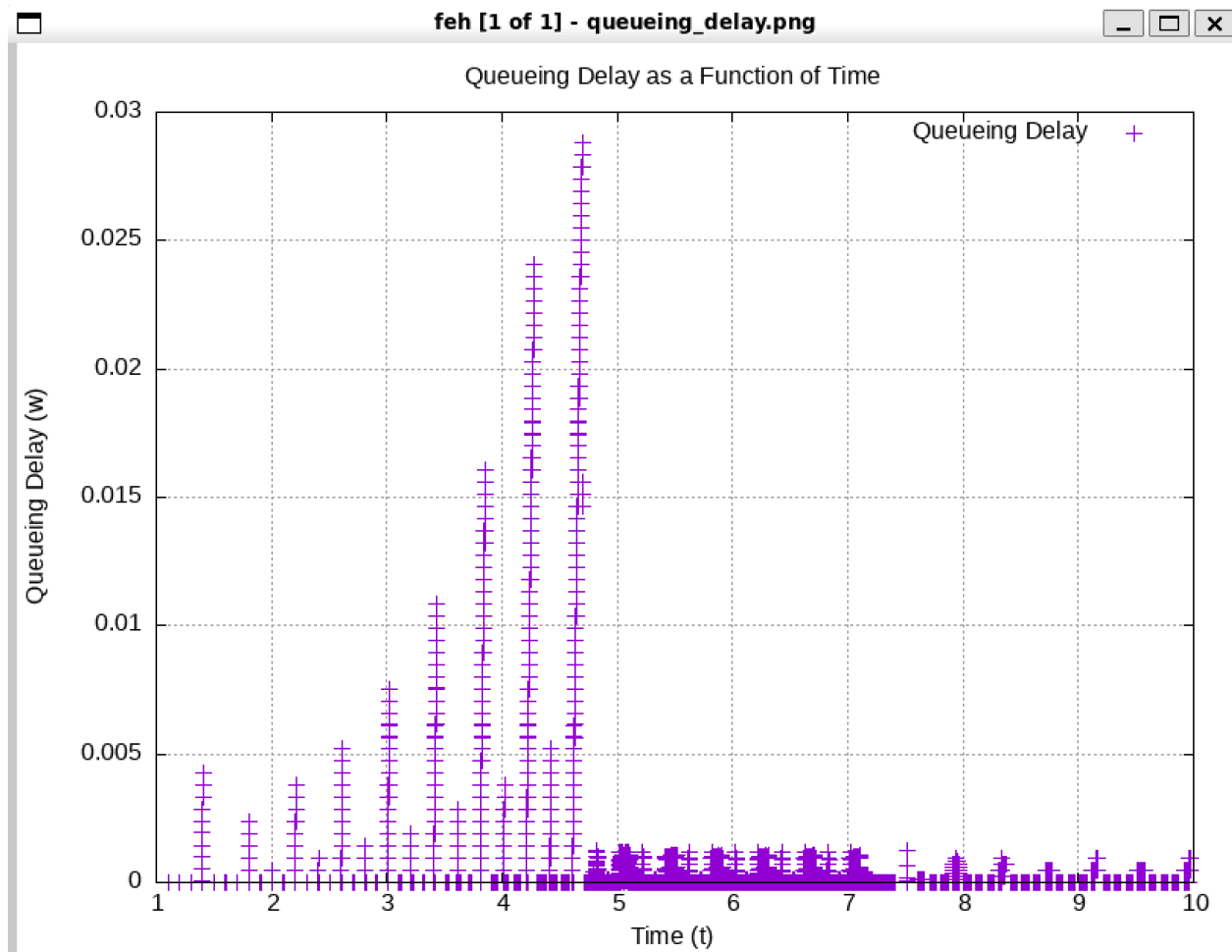
Congestion Avoidance Phase:

After time ≈ 6 , the cwnd starts to level off and oscillates around a steady value. This indicates the transition to the **congestion avoidance phase**, where cwnd growth slows down and becomes linear (or stops growing) due to congestion control mechanisms.

Packet Loss or Network Congestion:

The flat region (after time ≈ 7) suggests that cwnd is no longer increasing, likely due to **packet loss** or reaching the network's capacity. This behavior is consistent with TCP reducing or halting cwnd growth when signs of congestion appear.

c)



Low Delays Initially: Before 3 seconds, the queueing delay is minimal, reflecting smooth traffic flow with no signs of congestion.

Sudden Spike in Queueing Delay: Around 4 seconds, the queueing delay sharply increases, indicating a potential buildup of packets in the queue, likely due to a congestion event.

Stabilisation After Congestion: After 5 seconds, the queueing delay significantly decreases and stabilises, suggesting that the network recovered from congestion, possibly due to congestion control mechanisms like reducing the sending rate.

d)

Comparison:

The first graph refers to the graph of question 1.

1. **Queueing Delay Behavior:**

- In the **first graph**, the queueing delay peaks sharply around the 3-second mark and then stabilizes with smaller fluctuations. This indicates congestion likely occurring due to the bottleneck in the second link (Node1-Node2), which has lower bandwidth (7 Mbps) and lower latency (10 ms).
- In the **second graph**, the delay also rises significantly but peaks later (closer to the 5-second mark) and stabilizes more smoothly. The bandwidth and latency of both links are identical (10 Mbps and 100 ms), so the queueing delay accumulates more evenly.

2. **Maximum Queueing Delay:**

- The first graph shows a peak queueing delay of approximately 0.035 seconds, indicating severe congestion in the bottleneck link (Node1-Node2 with 7 Mbps).
- The second graph peaks at around 0.03 seconds, which is slightly lower. This suggests that having uniform bandwidth and latency across both links reduces the severity of the delay.

3. **Time for Delay to Stabilize:**

- In the first graph, the delay stabilizes after around 4 seconds.
- In the second graph, stabilization occurs slightly later, around 5 seconds.

4. **Impact of Network Bottleneck:**

- The first graph reflects the impact of a slower second link (7 Mbps), causing higher queueing delay as traffic accumulates more rapidly.
- The second graph benefits from uniform link properties, reducing overall congestion and making the queueing behavior more consistent.

Insights:

1. **Bottleneck Analysis:**

- The first configuration (Node1-Node2 at 7 Mbps) creates a bottleneck, leading to higher queueing delay as packets accumulate at Node1 before being transmitted at the slower rate. This demonstrates the importance of balancing bandwidth across links in a network path.

2. **Latency Impact:**

- In the second configuration, the increased latency (100 ms) for Node1-Node2 doesn't significantly affect the queueing delay because the bandwidth matches that of the first link, preventing congestion.